

Business Requirements Document (BRD) for Food Delivery Application

Project Title: Food Delivery Microservices System

Table of Contents

1. Introduction
 2. Objectives
 3. Scope
 4. Use Cases
 5. Non-Functional Requirements
 6. Conclusion
-

1. Introduction

The Food Delivery Microservices System aims to provide a seamless platform for customers to order food from local restaurants, track deliveries, and manage their accounts through a robust microservices architecture.

2. Objectives

- Enhance user experience through easy navigation and quick order placement.
- Enable restaurants to manage their menus and orders efficiently.
- Provide real-time tracking of deliveries.
- Facilitate secure payments and order history management.

3. Scope

This document outlines ten key use cases relevant to the food delivery domain that will be implemented using a microservices architecture. Each use case includes detailed descriptions and workflows.

4. Use Cases

Use Case 1: User Registration

- **Description:** Users can create an account on the food delivery platform.
- **Actors:** User, Registration Service
- **Preconditions:** The user provides valid personal information.
- **Postconditions:** A new user account is created, and the user receives confirmation.
- **Main Flow:**

1. User navigates to the registration page.
 2. User fills out the registration form with required information (name, email, phone number).
 3. Registration Service validates the information (checks for duplicates).
 4. If valid, the service creates a new user record in the database.
 5. The service sends a confirmation email or SMS to the user.
-

Use Case 2: Restaurant Search

- **Description:** Users can search for available restaurants based on location and cuisine.
 - **Actors:** User, Restaurant Search Service
 - **Preconditions:** The user is logged in or browsing as a guest.
 - **Postconditions:** The user receives a list of available restaurants.
 - **Main Flow:**
 1. User accesses the restaurant search page.
 2. User enters search criteria (location, cuisine type).
 3. Restaurant Search Service retrieves relevant restaurants from the database.
 4. The service formats the restaurant information (name, rating, delivery time) for display.
 5. The user views the list of available restaurants on the screen.
-

Use Case 3: View Menu

- **Description:** Users can view the menu of a selected restaurant.
 - **Actors:** User, Menu Service
 - **Preconditions:** The user has selected a restaurant.
 - **Postconditions:** The user views the menu items.
 - **Main Flow:**
 1. User selects a restaurant from the search results.
 2. Menu Service retrieves the restaurant's menu from the database.
 3. The service displays menu items (descriptions, prices, images).
 4. User can filter items (e.g., vegetarian, spicy).
 5. User selects items to add to their cart.
-

Use Case 4: Place Order

- **Description:** Users can place an order for selected menu items.
- **Actors:** User, Order Service, Payment Service
- **Preconditions:** The user has items in their cart.
- **Postconditions:** The order is submitted and a confirmation is sent to the user.
- **Main Flow:**
 1. User reviews items in their cart and proceeds to checkout.
 2. Order Service calculates the total cost (including delivery fee).
 3. User enters delivery address and selects a payment method.

4. Payment Service processes the payment securely.
 5. The Order Service confirms the order and sends a confirmation email or SMS to the user.
-

Use Case 5: Track Order

- **Description:** Users can track the status of their orders in real-time.
 - **Actors:** User, Tracking Service
 - **Preconditions:** The user has an active order.
 - **Postconditions:** The user is informed of the order status and delivery time.
 - **Main Flow:**
 1. User navigates to the "My Orders" section.
 2. Tracking Service retrieves the status of the user's current orders.
 3. The service displays the order status (preparing, out for delivery).
 4. User can view the estimated delivery time and driver information.
 5. The service updates the user with any status changes via push notifications or SMS.
-

Use Case 6: Review and Rate Order

- **Description:** Users can leave reviews and ratings for completed orders.
 - **Actors:** User, Review Service
 - **Preconditions:** The user has completed an order.
 - **Postconditions:** The review and rating are submitted and stored.
 - **Main Flow:**
 1. User navigates to the "My Orders" section.
 2. User selects a completed order to review.
 3. Review Service prompts the user for a rating (1 to 5 stars) and comments.
 4. User submits the review.
 5. The service stores the review and updates the restaurant's overall rating.
-

Use Case 7: Manage Account

- **Description:** Users can view and manage their account information.
- **Actors:** User, Account Management Service
- **Preconditions:** The user is logged into their account.
- **Postconditions:** The user's account information is updated.
- **Main Flow:**
 1. User logs into their account and navigates to the "Account Settings" section.
 2. Account Management Service retrieves the user's account details.
 3. User can edit personal information (name, email, password).
 4. User submits changes to update their account information.
 5. The service confirms the updates and notifies the user via email or SMS.

Use Case 8: Save Favorite Restaurants

- **Description:** Users can save their favorite restaurants for quick access.
- **Actors:** User, Favorites Service
- **Preconditions:** The user is logged into their account.
- **Postconditions:** The restaurant is added to the user's favorites list.
- **Main Flow:**
 1. User searches for restaurants and selects one to view.
 2. Favorites Service prompts the user to save the restaurant.
 3. User clicks the "Add to Favorites" button.
 4. The service stores the restaurant in the user's favorites list.
 5. The user receives confirmation that the restaurant has been added.

Use Case 9: Update Delivery Address

- **Description:** Users can update their delivery address in their profile.
- **Actors:** User, Address Management Service
- **Preconditions:** The user is logged into their account.
- **Postconditions:** The user's delivery address is updated in the system.
- **Main Flow:**
 1. User navigates to the "Account Settings" section.
 2. Address Management Service retrieves the user's current delivery address.
 3. User enters a new delivery address and submits the changes.
 4. The service validates the new address format.
 5. The service updates the address in the database and confirms the update.

Use Case 10: Customer Support Request

- **Description:** Users can submit support requests for assistance with orders.
 - **Actors:** User, Support Service
 - **Preconditions:** The user must be logged in or have an active order.
 - **Postconditions:** The support request is logged and acknowledged.
 - **Main Flow:**
 1. User navigates to the customer support section of the app or website.
 2. User fills out a support request form detailing the issue (order problem, refund request).
 3. Support Service logs the request in the database.
 4. The service assigns a support ticket and sends an acknowledgment to the user.
 5. A support representative contacts the user to resolve the issue.
-

5. Non-Functional Requirements

Overview

This section outlines the non-functional requirements for this application utilizing a microservices architecture implemented with Spring Cloud, MySQL, OAuth2, and JWT. These requirements are essential for ensuring that the application meets performance, security, and operational standards.

5.1. Scalability

- **Horizontal Scaling:** Each microservice must be designed to scale horizontally by deploying multiple instances based on demand. Load balancing must be implemented to distribute traffic evenly.
- **Service Discovery:** Utilize Spring Cloud Netflix Eureka for dynamic service discovery, enabling microservices to find and communicate with each other efficiently.

5.2. Performance

- **Response Time:**
 - The system should respond to user requests within 200 milliseconds for 95% of operations.
 - For batch processing tasks (e.g., bulk transactions), the maximum acceptable response time should not exceed 2 seconds.
- **Throughput:** Each microservice should handle a minimum of 100 transactions per second under normal load conditions.

5.3. Security

- **Data Encryption:**
 - Sensitive data (e.g., user credentials, personal information) must be encrypted in MySQL using AES-256 encryption.
 - All communication between microservices must be encrypted using TLS (Transport Layer Security).
- **Authentication and Authorization:**
 - Implement OAuth2 for user authentication, utilizing JWTs (JSON Web Tokens) for secure token-based access.
 - Tokens must have an expiration time of 15 minutes and should be refreshable without re-authentication for active sessions.

5.4. Availability

- **Uptime Requirement:** The system must achieve 99.9% availability, with scheduled maintenance windows communicated in advance to users.
- **Failover Mechanisms:** Implement circuit breaker patterns using Spring Cloud Resilience4j to prevent cascading failures and ensure resilience under load.

5.5. Compliance

- **Data Protection:** Ensure compliance with relevant data protection regulations such as GDPR and PCI DSS.
- **Audit Logging:** Maintain detailed logs of user activities, access events, and system changes for audit purposes. Logs should be retained for a minimum of 6 months.

5.6. Reliability

- **Error Handling:** Each microservice should have robust error handling and return appropriate HTTP status codes (e.g., 4xx for client errors, 5xx for server errors).
- **Data Consistency:** Implement eventual consistency for distributed transactions using patterns like Saga or CQRS (Command Query Responsibility Segregation).

5.7. Maintainability

- **Code Modularity:** Each microservice must be modular and follow the Single Responsibility Principle, making it easier to update and deploy independently.
- **Configuration:** Each microservice must be configured all external properties using Spring Cloud Config server.
- **CI/CD Pipeline:** Establish a Continuous Integration/Continuous Deployment (CI/CD) pipeline using tools like Jenkins or GitLab CI for automated testing and deployment of microservices.

5.8. Usability

- **API Documentation:** Provide comprehensive API documentation using Swagger/OpenAPI to facilitate ease of use for developers integrating with the microservices.
- **Error Messages:** Implement user-friendly error messages and codes to guide users in troubleshooting issues.

5.9. Monitoring and Logging

- **Centralized Logging:** Use a centralized logging solution (e.g., ELK Stack - Elasticsearch, Logstash, Kibana) for collecting and analyzing logs from all microservices.
- **Performance Monitoring:** Implement monitoring tools (e.g., Spring Boot Actuator, Prometheus, Grafana) to track system performance metrics, service health, and alert for anomalies.

5.10. Interoperability

- **Standard Protocols:** Ensure that all microservices communicate using RESTful APIs or gRPC, adhering to standard HTTP methods and response formats (JSON).
- **API Gateway:** Utilize Spring Cloud Gateway to provide a single entry point for all microservices, enabling routing, load balancing, and additional security measures.

6. Conclusion

The Food Delivery Microservices System aims to enhance operational efficiency and user experience through the outlined use cases. This architecture will facilitate rapid deployment, scalability, and an improved user experience.
